The Stress-Test Suite

38 real-world scenarios the agent must survive

Arkay Solutions · Marketing Automation Agent · Final Consolidated Edition

What this document is for

These are not bug reports or a QA checklist. They are realistic situations — a busy workshop night, a client who changes jobs, a subscriber who books a call from a dead email address — written to prove the system has been thought through end to end, and to give Devi concrete language to harden the build prompt BEFORE the code exists. Every scenario is derived from the final design spec: if the spec makes a rule, there is a scenario here that tries to break it.

How to read each scenario

A story (what really happens) → what the system should do → what a naive build would get wrong → the exact sentence to put in the prompt so it gets built right the first time. That last box is the deliverable: it turns testing into prevention.

Coverage Map — the variety guarantee

The suite deliberately spans **eight kinds of stress**, so it is systematic rather than a pile of edge cases. Every scenario belongs to at least one type; several belong to more.

Stress type	What it probes	Scenarios
Dirty data	Malformed, missing, or unexpected field values	5, 13, 14, 15, 20, 21, 33
Adversarial / security	Hostile input, public endpoints, injection	6, 12, 34
Volume & scale	Bursts, big imports, execution limits	1, 9, 23, 24
Concurrency & timing	Races, out-of-order events, simultaneous writes	1, 19, 23, 27, 28
Human behavior	Rachel and users doing normal, messy things	16, 17, 22, 30, 31
Dependency failure	Third parties down, slow, or lying	7, 25, 26, 34
Silent failure	Wrong outcome, success reported	3, 4, 22, 35, 36, 38
Catastrophic / recovery	Mass destruction with no undo	37

Why scenarios, not test cases

A test case asks: *“does this function handle a malformed email?”* A scenario asks: *“it's 8pm, forty people are signing up on their phones for a workshop starting in an hour, and three of them typo their address — what happens?”*

- **It's real.** These are conditions Arkay will actually be in.
- **It bundles failures.** One realistic moment stresses six spec rules at once — which is how systems actually break.
- **It writes the prompt for you.** “Handle bulk near-simultaneous registrations with typos and duplicates without losing anyone” is a sentence Devi can paste into the build instructions.

The Via Negativa framing: the point is not to add features — it is to subtract the assumption that it works. Each scenario is a bet that the agent will guess, and guess wrong. When the bet pays off, we don't file a bug: we write a sentence into the prompt so the guess never happens.

What “not brittle” means: a brittle system does the wrong thing and reports success. Nearly every serious scenario below fails quietly — a lead vanishes, a digest doesn't send, a newsletter reaches four people instead of three hundred — and the logs all say success. That is the thing this suite exists to prevent.

Data & Schema (spec §5, §8)

Scenario 1 — The workshop rush

It's 7pm Tuesday. Rachel's workshop starts at 8. Forty people register on Luma in the last hour, most on their phones, in a hurry. Three typo their email (jsmith@gmial.com). Two are already blog subscribers. One registers twice because the confirmation page was slow and she tapped again. One pastes their email with a trailing space. Next morning, Rachel has a 9am discovery call with someone from that list.

What should happen	Forty registrations become the correct number of contacts — not forty-three, not thirty-seven. The two existing subscribers gain a second source tag, not a duplicate row. The double-registration collapses into one. Typos are caught at verification. The trailing space is trimmed. By 8am the 9am contact is in the system with their event attendance visible on Rachel's digest.
What a naive build gets wrong	Duplicate rows for the two existing subscribers and the double-tapper, because check-and-write wasn't atomic during a burst. The typos burn verification credits. The 9am digest doesn't mention the workshop, so Rachel walks in not knowing they just attended.

→ Language for Devi's prompt:

```
Contact identity is the lowercased, trimmed email. Before ANY write,
look up by that key and UPDATE if found; never INSERT a second row.
Wrap lookup-and-write in a lock so simultaneous registrations cannot
both pass the "does this exist?" check.

Validate email FORMAT before calling the verification API, so
obviously-malformed addresses never consume a paid credit.

The pre-call digest must include which events the contact attended.
```

Scenario 13 — The person with no name

Someone subscribes with only an email address; the name fields were optional and they skipped them. Two weeks later the newsletter goes out, personalized with "Hi {{FNAME}}."

What should happen	They receive "Hi there," or the greeting gracefully omits the name. The contact is valid — email is the only required field.
What a naive build gets wrong	The email reads "Hi ," or worse, ships the raw {{FNAME}} merge tag. Rachel's editorial brand goes out broken.

→ Language for Devi's prompt:

```
Every field except email is optional and may be blank. Any template
using a name must have a fallback ("Hi there" when empty). Never
render an empty merge tag or a raw {{TAG}} into a sent email.
```

Scenario 14 — The phone number that isn't a phone number

A workshop attendee types "call me on WhatsApp" into the phone field. Another types "+44 (0) 20 7946 0958". Another, "555-1234 ext 22".

What should happen	None of them break the contact. Phone is optional and cosmetic — it must never be the reason a lead is rejected.
What a naive build gets wrong	Phone normalization throws on unparseable input and the whole row fails, losing a lead over a field nobody uses.

→ Language for Devi's prompt:

```
The phone field is optional and free-form. Store whatever the user
typed. Never reject a contact because the phone could not be parsed.
No field except email may cause a row to be rejected.
```

Scenario 15 — The tag list that grows forever

A contact who's been around a year: came via the blog, attended four workshops, was added from LinkedIn, moved through five lifecycle states.

What should happen	One status tag, one tag per source (not one per signup), one per event attended. The field stays readable.
What a naive build gets wrong	source:luma appears four times. Old status tags never got cleaned up. Every segment built on tags is now unreliable.

→ Language for Devi's prompt:

```
Tags are a SET, not a list. Adding an existing tag changes nothing.
Store them sorted and de-duplicated. Source tags appear at most once
each, no matter how many times that source sends the contact.
```

Scenario 5 — The same person, four spellings, six months apart

Jane subscribes as jane.smith@acme.com. Four months later she registers for a workshop as Jane.Smith@Acme.com. Rachel adds her from a LinkedIn export as JANE.SMITH@ACME.COM with a new job title. Then she signs up from her phone with a trailing space. Meanwhile her company changed — LinkedIn says "Beta Corp," the blog record says "Acme."

What should happen	One contact. Four source tags. Job title updates. When the company field conflicts, the newer value wins but the old value goes to an audit log. Nothing is lost silently.
What a naive build gets wrong	Four separate contacts, all the same human. Jane gets four copies of every newsletter. Or the LinkedIn import overwrites her real name with a badly-cased one and silently clobbers "Acme."

→ Language for Devi's prompt:

```
Match contacts case-insensitively on the trimmed, lowercased email.
Merge, never duplicate.

On merge: a blank incoming value NEVER overwrites an existing value.
When both exist and differ, the newer wins and the old value is
written to an audit log.

LinkedIn imports may ADD tags and fill EMPTY fields, but must never
overwrite a field that already has a value.
```

Scenario 20 — The merge that erases a job title

A contact came from the blog with "Senior Director." Six months later they register for a workshop on Luma, whose form doesn't ask for job title — so that field arrives blank.

What should happen	They keep "Senior Director." A blank incoming value never overwrites populated data.
What a naive build gets wrong	The Luma record overwrites the whole row and the job title is silently erased. Repeat across a year of workshops and the database slowly empties itself of everything the blog form collected.

→ Language for Devi's prompt:

```
On merge, a blank or missing incoming value NEVER overwrites an
existing populated value. Update field-by-field, not by replacing the
whole record. Sources that collect fewer fields must not erase data
collected by sources that collect more.
```

Scenario 21 — Two records, both partially right

The blog record says company "Acme," no phone. The LinkedIn import says "Acme Corporation," with a phone. Neither is blank. They're just different.

What should happen	Newer wins (LinkedIn), old value to the audit log, phone filled in since it was blank. Nothing lost silently.
What a naive build gets wrong	Either LinkedIn is rejected wholesale (so the phone is never filled), or it overwrites everything with no record of what was there. The audit log is what makes an overwrite reversible.

→ Language for Devi's prompt:

```
When both existing and incoming values are populated and differ, the
newer wins — and the previous value is written to an audit log with a
timestamp. An overwrite must always be recoverable. Apply this
field-by-field, so a conflict in one field does not affect others.
```

Lifecycle State (spec §6)

Scenario 2 — The client who books from a dead address

Someone subscribed six months ago. Their company email bounced last month when they left the job, so the system marked them invalid and stopped mailing them. Today they book a discovery call — through the Calendar link, but the invite still lists their old work email as the guest.

What should happen	Rachel gets her pre-call digest. Full stop. A real person booked a real call. The bounce record should suppress marketing email — never Rachel's awareness of a booking.
What a naive build gets wrong	The agent sees "invalid," concludes the contact is dead, and skips the digest. Rachel walks into a call blind. This is the worst failure in the system: silent, invisible, and it costs a client.

→ Language for Devi's prompt:

```
A booking always wins over an email-deliverability flag. If a booked
contact is marked invalid, still set them to "booked" and still send
Rachel the pre-call digest. Keep the suppression flag so bulk email
stays blocked, but never let a bounce record suppress a digest.

If the calendar guest email matches no contact at all, create a
minimal contact record and send the digest anyway.
```

Scenario 3 — The brand-new lead who gets deleted

Someone subscribes on the 28th. The monthly maintenance job runs on the 1st — three days later. They have zero opens and zero clicks, because no newsletter has gone out yet. Nothing has ever been sent to them.

What should happen	Nothing happens to them. They are three days old. The "stale" rule exists to find people who went quiet after being engaged with, not people who have never been contacted.
What a naive build gets wrong	The job checks "zero engagement in 60 days," which is trivially TRUE for someone never emailed. The brand-new lead is tagged stale, re-verified, and archived — before their first newsletter. The system quietly deletes new leads every month, and every happy-path test passes.

→ Language for Devi's prompt:

```
A contact can only be marked stale if they have BOTH no engagement in
60 days AND were created more than 60 days ago. A contact who has
never been sent an email can never be stale. Check the creation date,
not just the engagement date.
```

Scenario 16 — Rachel replies to the digest three days late

She gets a digest with WON / LOST links. The call happens Tuesday. She's slammed and doesn't click until Friday — by which point she's had two more calls and forgotten which prospect this was. She clicks WON anyway.

What should happen	The link carries the contact's identity, so the system knows exactly who "won" refers to regardless of when she clicks.
What a naive build gets wrong	The system applies the outcome to "the most recent booking" — marking a prospect she lost as won, and leaving the real client in the nurture flow.

→ Language for Devi's prompt:

```
The outcome links in the pre-call digest must encode the specific
contact and event they refer to. Never infer the target of an outcome
from recency or from "the most recent booking." Rachel may respond
days late or out of order.
```

Scenario 17 — Rachel clicks WON twice

She clicks WON, isn't sure it registered, clicks again. Then replies to the email with a note.

What should happen	Marked won, once. The second click changes nothing. Her prose doesn't confuse the parser.
What a naive build gets wrong	The second click triggers a won-to-won transition, which isn't in the transition table, and errors. Or the parser chokes on her free text and logs a failure nobody sees.

→ Language for Devi's prompt:

```
Outcome logging must be idempotent: clicking WON twice has the same
effect as once. Parse only the structured part of the reply and ignore
additional prose. A repeated outcome is not an error.
```

Scenario 18 — Identity is email, and only email

A client Rachel won last year is now at a new company and subscribes again from a new address. The old record is still marked won and terminal.

What should happen	The new address is a genuinely new contact. The old won record stays as history.
What a naive build gets wrong	The agent tries to be "helpful" and merges them on name similarity — corrupting both records.

→ Language for Devi's prompt:

```
Identity is the email address, and nothing else. Never merge two
contacts based on matching names, companies, or any fuzzy similarity.
Two different email addresses are two different people, even if
everything else matches.
```

Tags & Send Integrity (spec §7)

Scenario 19 — The tag applied mid-send

The weekly newsletter is mid-send, working through 300 contacts. At contact #150, someone books a call — changing that contact's status from active to booked while they're in the send segment.

What should happen	No crash. The send completes for all 300. Either they get the newsletter (they were active when the segment was built) or they don't — but nobody after them is affected.
What a naive build gets wrong	The segment is re-evaluated mid-loop, the contact no longer matches, the iteration breaks — silently dropping contacts #150–300 from the send.

→ Language for Devi's prompt:

```
Build the recipient list ONCE at the start of a send and iterate over
that fixed snapshot. Do not re-query the segment mid-send. A contact
```

```
whose status changes during a send must not break the loop or cause
remaining contacts to be skipped.
```

Scenario 10 — The prospect who no-shows, then reschedules, then buys

They book, don't show. Two weeks later they book again, apologetic. This time they show, the call goes well, Rachel wins the client. The following month, the client is still receiving the prospect newsletter.

What should happen	No-show returns them to active. The second booking sends a FRESH digest — including the no-show, which is useful context. When Rachel marks them won, they exit the nurture flow.
What a naive build gets wrong	The second digest never sends, because the contact was already marked "digested." And after the win, they still carry status:active alongside won, so a paying client keeps getting cold-lead marketing.

→ Language for Devi's prompt:

```
Track pre-call digests per CALENDAR EVENT, not per contact. A contact
who books a second time must receive a second digest. Rescheduling an
event must re-arm its digest.

A contact has exactly ONE lifecycle status at a time. When status
changes, remove the old status tag in the same operation that adds the
new one. A "won" client must not still be tagged "active."
```

Ingestion (spec §9)

Scenario 22 — The Google Form question Rachel renames

Three months in, she edits the blog form and changes "Job Title" to "What's your role?" because it sounds friendlier.

What should happen	The system either keeps working, or fails LOUDLY — logging "unmapped question" so someone notices in a day, not a quarter.
What a naive build gets wrong	The job title field silently stops being populated. Nobody notices for months. Every contact from that point has a blank role, and the data quietly degrades.

→ Language for Devi's prompt:

```
Map form questions by title. If a question title does not match any
known mapping, log it explicitly as an unmapped field and alert.
Silently ignoring an unrecognized question is not acceptable — someone
editing the form must not be able to break data collection without
anyone noticing.
```

Scenario 23 — Two workshops on the same night

Rachel runs two Luma events in the same week. Both export CSVs into the same Drive folder within minutes. One person attended both.

What should happen	Both files process. The person who attended both gets two event tags on one record. Both files move to Done and neither is re-processed.
What a naive build gets wrong	The job grabs both files and processes them concurrently, hitting the same contact twice at once — creating a duplicate. Or one file's event tag overwrites the other's, and Rachel loses the record of which workshops someone attended.

→ Language for Devi's prompt:

```
Process import files one at a time, not concurrently. A contact
appearing in multiple import files must accumulate all their event
tags on one record. After a file is processed, move it so it can never
be picked up again — even if the job is interrupted between processing
and moving.
```

Scenario 24 — The CSV that's already been processed

The Luma job processes a CSV, but the script is killed by the 6-minute limit AFTER importing rows but BEFORE moving the file to Done. The next run picks up the same file.

What should happen	Re-processing changes nothing — every row is an upsert. No duplicates. The file then moves to Done.
What a naive build gets wrong	Every row is inserted again. Forty contacts become eighty. This is the failure that makes "idempotent" more than a buzzword.

→ Language for Devi's prompt:

```
Assume any job may be interrupted between doing the work and marking
the work done. Every import must be safely repeatable: re-processing
the same file must produce exactly the same result as processing it
once. Never assume the "mark as done" step succeeded.
```

Scenario 9 — The 3,000-row conference list

Rachel comes back from a conference with a 3,000-row attendee export. Two minutes into the import, Apps Script hits its six-minute limit and the script is killed mid-file.

What should happen	The import saves its place, resumes on the next run, completes every row. No contact skipped, none imported twice.
What a naive build gets wrong	The script dies at row 1,400. Nobody's sure which rows made it. Re-running duplicates the first 1,400. Or it reports "complete" having processed half the file — and 1,600 leads from a conference Rachel paid to attend simply never exist.

→ Language for Devi's prompt:

```
All bulk jobs must assume they will be interrupted by the platform's
execution time limit. Save a cursor after each batch, resume from the
cursor on the next run, and design every operation to be safely
repeatable so a resume never double-processes a row.

Never report success on a job that did not finish.
```

Verification (spec §10)

Scenario 7 — The verification service has a bad morning

ZeroBounce goes down for two hours on a Wednesday. During that window, eleven people subscribe from a LinkedIn post that's doing well. The API returns 500s, then times out, then reports "quota exceeded" because free credits ran out.

What should happen	All eleven contacts are SAVED, marked risky, held back from bulk sends, queued for re-verification. Rachel gets an alert that the quota is nearly exhausted, with the next tier's price, and decides.

What a naive build gets wrong	The intake fails because verification failed, and eleven real leads from a successful post are silently dropped. A third-party outage destroys leads — precisely the "lead leakage" this project exists to eliminate.

→ Language for Devi's prompt:

```
Never lose a contact because a third-party service failed. If email
verification times out, errors, or runs out of credits: still save the
contact, mark it "risky," hold it from bulk sends, queue it for
re-verification later.

Google Sheets is the source of truth. A contact must be written to the
sheet even if every downstream service is unreachable.

When a free-tier limit reaches 90%, email Rachel the current usage, the
ceiling, and the next tier's price. Never auto-upgrade.
```

Scenario 25 — The catch-all domain

Someone subscribes from a company that accepts all mail. The verifier can't say whether the mailbox exists. It returns "unknown."

What should happen	Marked risky — kept, synced, held from bulk sends until there's more signal. Not treated as valid, not thrown away.
What a naive build gets wrong	The code has an if-valid/else-invalid shape, so "unknown" falls into the else — and a legitimate lead at a large company is marked undeliverable and suppressed forever.

→ Language for Devi's prompt:

```
Email verification returns FOUR outcomes, not two: valid, invalid,
risky/unknown (catch-all domains), and do-not-mail. Handle all four
explicitly. Never write code that treats "not valid" as "invalid" —
"unknown" is a real and common result for legitimate corporate
addresses.
```

Scenario 26 — The re-verification loop

A contact was marked risky in March. The monthly job re-verifies. Still risky. Next month, same. This continues for a year.

What should happen	Either they resolve, or the system stops burning a paid credit on the same unresolvable address every month.
What a naive build gets wrong	Twelve credits spent on one contact, out of a free tier of ~100/month. Multiply by a few dozen risky contacts and the whole quota is consumed re-checking dead ends.

→ Language for Devi's prompt:

```
Do not re-verify the same address indefinitely. After a small number of
consecutive "risky" results (e.g. 3), stop re-verifying and leave the
contact in its current state. Verification credits are finite and must
not be spent repeatedly on the same unresolvable address.
```

Booking & Digest (spec §11)

Scenario 8 — The supplier meeting that looks like a sales call

Rachel puts an event on her calendar: "Recall the vendor — pricing question," and invites her contact at the printing company. It's an internal errand. An hour before it starts, she gets a pre-call digest about her printer.

What should happen	Nothing. This is not a discovery call and the system should know that.
What a naive build gets wrong	The booking rule matches any title containing "call" — and "Recall" contains "call." So does "Callback." Every internal meeting with an external guest generates a phantom contact and a bogus digest. Rachel starts ignoring digests. Once she does, the feature is dead — and the next one she ignores is a real client.

→ Language for Devi's prompt:

```
A booking is only a discovery call if the event title STARTS WITH a
specific marker (e.g. "Discovery Call"), not merely contains the word
"call" anywhere. Beware substring matches: "Recall" and "Callback"
both contain "call."

Precision matters more than recall here. A missed booking is visible.
A false booking is noise that destroys trust in every digest after it.
```

Scenario 27 — The call booked 40 minutes from now

Someone books at 9:20am for a 10:00am slot. The digest is supposed to arrive "at least one hour before."

What should happen	Rachel gets the digest IMMEDIATELY. The system recognizes it can't hit the one-hour target and sends anyway, right now.
What a naive build gets wrong	The scheduler only scans for events starting 60–75 minutes out. This booking is 40 minutes out, falls outside the window, and is never digested at all — and the closer the booking, the more likely she needed the prep.

→ Language for Devi's prompt:

```
If a booking is made less than an hour before the call, send the
pre-call digest IMMEDIATELY rather than skipping it. The scan must
catch events starting sooner than the target lead time, not only those
inside a narrow future window. A late digest is always better than no
digest.
```

Scenario 28 — The call booked three months out

Someone books a call in March. It's December. The 15-minute polling job runs every 15 minutes for the next three months.

What should happen	State set to booked now; digest fires an hour before the call in March. Nothing expensive happens in between.
What a naive build gets wrong	The job re-processes the same far-future booking thousands of times, burning quota. Or it digests immediately in December, three months early.

→ Language for Devi's prompt:

```
Detecting a booking and sending its digest are two separate things with
two separate timings. Set the contact's state when the booking is made;
send the digest an hour before the event, which may be months later.
The polling scan must be bounded — only look at events in the near
future, not the entire calendar.
```

Scenario 29 — The all-day event

Rachel blocks out "Discovery Call — offsite" as an all-day event, with the prospect invited.

What should happen	Handled sensibly or ignored — but it doesn't crash and doesn't send a digest at a nonsensical time.
What a naive build gets wrong	An all-day event has no start TIME, so "one hour before the start" produces garbage — a digest at midnight, or an error that kills the polling job for every other booking in the same run.

→ Language for Devi's prompt:

```
Handle all-day calendar events explicitly: they have a date but no
meaningful start time. Do not attempt to compute "one hour before" for
them. Never let one malformed event kill the polling job for all the
other events in the same run.
```

Scenario 30 — Booked, cancelled, rebooked

Booked Monday. Cancelled Monday night. Rebooked Tuesday for a different slot.

What should happen	State goes booked → active → booked. The digest fires for the new slot. Rachel never gets a digest for a call that isn't happening.
What a naive build gets wrong	The cancellation isn't detected at all — the spec covers cancel/decline, but only if the agent implements the event-deleted case, which is easy to miss. Rachel gets a digest for a cancelled call, shows up, and nobody's there.

→ Language for Devi's prompt:

```
Handle event deletion and cancellation, not just creation and update.
A deleted or declined booking must return the contact to their prior
state and cancel any pending digest. Rachel must never receive a digest
for a call that has been cancelled.
```

Newsletter (spec §12)

Scenario 4 — The newsletter that quietly dies after week one

Rachel publishes a post. The newsletter goes out Thursday, beautifully, to all 300 subscribers. The next week she publishes another. Thursday comes. The newsletter "sends" — logs say success, Rachel gets her confirmation — but only the 4 people who joined in the last seven days actually receive it. The other 296 get nothing. Nobody notices for a month.

What should happen	All 300 active subscribers receive week two. And week three. And every week after.
What a naive build gets wrong	The send mechanism enters people into an automation using a trigger that fires only ONCE per person, ever. Week one works perfectly, which is exactly what makes this so dangerous — a single test send PASSES while the system is fundamentally broken.

→ Language for Devi's prompt:

```
The weekly newsletter must be able to reach the SAME subscriber every
week, repeatedly. Verify this by sending to a test contact twice in a
row and confirming they receive it both times — a single successful
send does NOT prove the mechanism works.

If the send mechanism can only enter a given person once, it is the
wrong mechanism. Use a trigger that re-fires (add a tag, send, then
remove the tag so it can be re-added next week).
```

Scenario 11 — The blog post scheduled for next month

Rachel drafts a post and schedules it for three weeks out. It sits in the feed with a future publication date. Thursday's newsletter job runs.

What should happen	The job sees no post published in the last seven days, logs "skipped," sends nothing.
What a naive build gets wrong	The check is "published date is within the last 7 days" — which is technically TRUE of a post dated three weeks in the FUTURE. The unfinished draft is emailed to all 300 subscribers, three weeks early.

→ Language for Devi's prompt:

```
A post counts as new only if its publication date is within the last
seven days AND is not in the future. Bound the check on both sides.
```

Scenario 31 — Two blog posts in one week

Rachel gets on a roll and publishes Monday and again Wednesday. Thursday's job runs.

What should happen	Both posts reach subscribers, or the skip is logged loudly. (This is a genuine spec gap — see Gap 7.)
What a naive build gets wrong	The spec says "the newest item," so Wednesday's post sends. Monday's post is never emailed — it's not the newest, and next week it'll be older than 7 days. It silently ages out. Rachel assumes all her posts go out.

→ Language for Devi's prompt:

```
Decide explicitly what happens when more than one post is published in
the same week. Either send all unsent posts, or send the newest and log
```

```
clearly that the others were skipped. Do not let a published post
silently never reach the newsletter.
```

Scenario 32 — The newsletter with zero eligible recipients

Every contact is stale, unverified, or invalid. The segment comes back empty. A new post is published.

What should happen	Logged as "no eligible recipients," no send attempted, no error — and Rachel is told, because an empty audience is a signal something is wrong.
What a naive build gets wrong	The system tries to send to an empty segment, the API errors, the job is marked failed — so it retries next week, fails again, and the error log fills with noise that masks real failures.

→ Language for Devi's prompt:

```
Handle the empty-recipient case explicitly: if no contacts are eligible
for a send, log it and exit cleanly. Do not treat an empty audience as
an error, but DO notify Rachel — an empty list is a signal that
something upstream is broken.
```

Scenario 33 — The post with no content

Rachel publishes a post that's just a title and an image — the RSS description is empty. Or the description contains raw HTML from her editor.

What should happen	The newsletter still builds. An empty summary means a shorter email, not a broken one. HTML is handled, not double-escaped into visible tag soup.
What a naive build gets wrong	Subscribers see <p> literally in their inbox. Rachel's editorial brand takes the hit.

→ Language for Devi's prompt:

```
Blog post fields may be empty or may contain HTML. The newsletter
template must render correctly in both cases: an empty summary produces
a shorter email, not a broken one; HTML in the source must not appear
as visible escaped tags in the sent email.
```

Security & Hostile Input (spec §5, §13)

Scenario 6 — The bored person on the blog form

Someone — a competitor, a bored teenager, a security researcher — fills in the blog form. In Company they type =cmd|'/c calc'!A1. In First Name they type <script>alert(1)</script>. Later that week, Rachel opens a pre-call digest.

What should happen	Both values land as harmless text. The digest displays them as literal characters. Nothing executes anywhere.
What a naive build gets wrong	The company name is written raw into Sheets, where a leading = makes it a FORMULA. The name is injected raw into the HTML digest, where <script> becomes live code in Rachel's inbox. The spec never mentions sanitizing — so an agent building from it almost certainly won't add it.

→ Language for Devi's prompt:

```
Sanitize every contact field before storing or displaying it.
Before writing to Sheets: if a value begins with =, +, -, or @, prefix
```

```
it with a single quote so it is stored as text, not a formula.
Before rendering into any HTML email: HTML-escape every field value.
Never trust a value that came from a form.
```

Scenario 12 — The public endpoint nobody thinks about

To receive open/click data, the system exposes a web endpoint anyone on the internet can reach. Someone finds it and posts a batch of fake "unsubscribed" events for Rachel's best contacts.

What should happen	Rejected — not cryptographically signed. Nothing changes. The attempt is logged.
What a naive build gets wrong	Unsigned requests are processed as real. A stranger can unsubscribe Rachel's entire list. The endpoint HAS to be public for the platform to reach it — the signature check is the ONLY thing protecting it, and it's exactly the detail an agent skips when nobody asks explicitly.

→ Language for Devi's prompt:

```
The webhook endpoint is publicly reachable, so every incoming request
must be verified with an HMAC signature against a shared secret before
it is processed. Reject and log anything unsigned or mismatched.

Ignore duplicate events (dedupe on event id) — the platform may deliver
the same event more than once.
```

Scenario 34 — The webhook that arrives before the contact exists

A contact is created in EmailOctopus and a webhook fires about them — but the Sheet write hasn't finished. The webhook arrives for a contact the Sheet doesn't know about.

What should happen	Handled gracefully — queued for retry or logged and skipped. No crash.
What a naive build gets wrong	The handler looks up the contact, finds nothing, throws — killing the whole webhook batch, including the 999 other valid events in the same request.

→ Language for Devi's prompt:

```
A webhook may arrive for a contact that does not exist locally yet.
Handle this without erroring. One bad event in a batch must never
prevent the other events in the same batch from being processed —
process each event independently and log failures individually.
```

Failure, Maintenance & Silence (spec §14, §15)

Scenario 35 — The weekly report with nothing in it

A quiet week. No sends, no opens, no bookings. Monday's report job runs.

What should happen	Rachel gets a report saying "quiet week — 0 sends, 0 new contacts, list steady at 312." The report ARRIVES, because its arrival is itself the signal the system is alive.
What a naive build gets wrong	The job errors on a division by zero (open rate = opens ÷ delivered, where delivered = 0), or sends nothing at all — so Rachel's only heartbeat goes silent, and she can't distinguish "quiet week" from "the system died."

→ Language for Devi's prompt:

```
The weekly report must ALWAYS send, even when every metric is zero.
Guard against division by zero when computing rates (0 opens out of 0
delivered is 0%, not an error). The report's arrival is Rachel's proof
the system is alive — silence must never be ambiguous.
```

Scenario 36 — The error log that nobody reads

Over six weeks, 340 rows quietly accumulate in the Error Log sheet. Nobody has looked. Among them are 12 real leads that failed to import.

What should happen	The weekly report includes an error count, so a growing error log is VISIBLE rather than something you have to remember to check.
What a naive build gets wrong	Errors are logged dutifully and never surfaced. The log becomes a graveyard. "We have logging" becomes a false comfort.

→ Language for Devi's prompt:

```
Logging an error is not the same as surfacing it. The weekly report
must include the count of new errors that week and the number of
unresolved errors total. A growing error log must be visible without
anyone remembering to open a spreadsheet.
```

Scenario 37 — The maintenance job that archives everyone ⚠

The webhook receiver has been silently failing for two months, so no opens or clicks have been recorded for anyone. The monthly maintenance job runs and finds that every single contact has "no engagement in 60 days."

What should happen	The system notices something is catastrophically wrong BEFORE archiving the entire list. A sanity check: if more than a set share of the audience would be archived in one run, stop and alert instead.
What a naive build gets wrong	The job dutifully archives all 300 contacts. Rachel's entire audience is gone, the cause is an unrelated bug two months old, and there is no undo.

→ Language for Devi's prompt:

```
Add a safety limit to any bulk destructive operation. If a single
maintenance run would archive more than a defined share of the audience
(e.g. 20%), STOP, change nothing, and alert Rachel — a mass archive
almost certainly means an upstream tracking failure, not that every
contact went cold at once.
```

Scenario 38 — The trigger that stops firing

Google silently disables an Apps Script trigger after too many consecutive failures — real, documented behavior. The newsletter trigger dies. No newsletter goes out for three weeks.

What should happen	Someone notices, because the weekly report shows when each job last ran.
What a naive build gets wrong	The system's own alerting depends on the system running — so when it stops, so does the alerting. Total silence looks exactly like a quiet week.

→ Language for Devi's prompt:

```
The system cannot rely on itself to report its own death. Include a
heartbeat: record the timestamp of each scheduled job's last successful
run, and have the weekly report show when each job last ran. If a job
has not run in longer than its interval, say so loudly. Silence must
```

```
never be indistinguishable from health.
```

The Consolidated Prompt Block

Everything above, distilled into one block Devi can paste directly into the build prompt as a “Robustness Requirements” section. Every line exists because a realistic scenario proved an agent would otherwise guess wrong.

How to use it
Devi pastes this into the build instructions. Nothing here is hypothetical — every line traces to a numbered scenario above.

```
ROBUSTNESS REQUIREMENTS (non-negotiable)

IDENTITY & DUPLICATES
- Contact identity is the lowercased, trimmed email address, and
  nothing else. Never merge on name/company similarity.
- Before any write, look up by that key and UPDATE if found. Never
  create a second row for the same address.
- Wrap lookup-and-write in a lock: two simultaneous signups for the
  same person must not both pass the "does this exist?" check.

DATA PRESERVATION
- A blank incoming value NEVER overwrites a populated one. Merge
  field-by-field, never by replacing the whole record.
- When two populated values conflict, newer wins AND the old value
  goes to an audit log. Every overwrite must be recoverable.
- LinkedIn imports may add tags and fill empty fields only.
- Tags are a set: adding an existing tag does nothing. Source tags
  appear at most once regardless of how often that source fires.

NEVER LOSE A CONTACT
- Google Sheets is the source of truth. Write the contact there even
  if every downstream service is unreachable.
- If verification fails (timeout, error, out of credits): still save
  the contact, mark it "risky," hold from bulk sends, queue for
  retry. A third-party outage must never destroy a lead.
- Validate email FORMAT before calling the paid verification API.
- Verification returns FOUR outcomes: valid, invalid, risky/unknown,
  do-not-mail. Never treat "not valid" as "invalid."
- Stop re-verifying an address after 3 consecutive "risky" results.
- No field except email may cause a row to be rejected.

SECURITY — TREAT ALL INPUT AS HOSTILE
- Before writing to Sheets: if a value starts with =, +, -, or @,
  prefix with a single quote so it stores as text, not a formula.
- Before rendering into any HTML email: HTML-escape every field.
- The webhook endpoint is public: verify an HMAC signature on every
  request; reject and log anything unsigned. Dedupe on event id.

LIFECYCLE CORRECTNESS
- A contact can only go stale if it has no engagement in 60 days AND
  was created more than 60 days ago. Someone never emailed can never
  be stale.
- A contact has exactly ONE status at a time. Remove the old status
  tag in the same operation that adds the new one. A "won" client
  must not still be tagged "active."
- Only defined state transitions are allowed; refuse and log anything
  else.
- Outcome logging is idempotent; outcome links encode the specific
  contact and event, never "the most recent booking."
```

```
BOOKINGS
- A booking always outranks an email-deliverability flag. If a booked
  contact is marked invalid, still send Rachel the pre-call digest.
- If a calendar guest matches no contact, create a minimal record and
  send the digest anyway.
- A discovery call is an event whose title STARTS WITH the agreed
  marker — not one that merely contains "call" ("Recall",
  "Callback").
- Track digests per EVENT, not per contact: a second booking gets a
  second digest; rescheduling re-arms it.
- A booking made less than an hour out gets its digest IMMEDIATELY,
  not never.
- Detecting a booking and sending its digest are separate events with
  separate timings.
- Handle event cancellation and deletion, not just creation.
- Handle all-day events, which have no meaningful start time.
- The digest must include which events the contact attended.

NEWSLETTER
- The weekly send must reach the SAME subscriber every week,
  repeatedly. Prove it by sending to a test contact twice. A single
  successful send does NOT prove the mechanism works.
- A post is new only if published within the last 7 days AND not in
  the future.
- Never send the same post twice.
- Handle the empty-recipient case cleanly, and notify Rachel.
- Post summaries may be empty or contain HTML; the template must
  render correctly either way.
- Every field except email may be blank. Templates using a name need
  a fallback. Never render an empty or raw merge tag.

DEFENSIVE ITERATION & BULK JOBS
- Build any recipient list ONCE as a snapshot; never re-query
  mid-send.
- One bad record in a batch must never stop the rest of the batch.
- Assume every long job will be killed by the 6-minute execution
  limit. Save a cursor, resume from it, and make every operation
  safely repeatable so a resume never double-processes.
- Never report success on a job that did not finish.
- Assume interruption between doing work and marking it done.

CIRCUIT BREAKERS
- If any single maintenance run would archive more than 20% of the
  audience, STOP, change nothing, and alert. A mass archive means an
  upstream failure, not that everyone went cold at once.

NEVER LET SILENCE LOOK LIKE HEALTH
- Record the last successful run time of every scheduled job. The
  weekly report shows when each last ran. A job that has not run in
  longer than its interval must be reported loudly.
- The weekly report ALWAYS sends, even when every metric is zero.
  Guard all rate calculations against division by zero.
- The weekly report includes new-errors-this-week and total
  unresolved errors. Logging an error is not the same as surfacing
  it.
- Fail loudly, never silently. A rejected row with an error is
  healthy; a swallowed row with a success message is the dangerous
  case.
```

Spec Gaps — Decisions Needed

Eight places where the spec doesn't say what should happen — meaning the agent *will* guess. Each has a recommended call, so this is a decision to approve rather than a problem to solve.

#	Gap	Why it matters	Recommended call
1	No field sanitization (Sc. 6)	Formula injection into the master sheet; script injection into Rachel's digest. Both trivially triggered from a public form.	Escape leading = + - @ before Sheets; HTML-escape before rendering into email.
2	Bounced contact books a call (Sc. 2)	The state model has no path for it. The bad guess means Rachel walks into a real call blind.	Booking wins. Digest sends. Bulk suppression stays in place.
3	No write locking (Sc. 1, 23)	The merge rule doesn't say check-and-write must be atomic. A workshop rush creates duplicates.	Use a lock around lookup-and-write.
4	Cleaning applied on one path only (Sc. 5)	Emoji/control-char stripping is specified for LinkedIn import but not the blog form — so one person can exist twice, differently.	Make normalization universal across all sources, defined once in the schema.
5	Future-dated posts (Sc. 11)	“Within last 7 days” is TRUE of a post dated next month. An unfinished draft mails to 300 people.	Bound the date check on both sides.
6	Booking regex too loose (Sc. 8)	“Recall” and “Callback” contain “call.” Phantom digests erode trust until Rachel ignores them all.	Require a title prefix or an explicit marker.
7	Multiple posts in one week (Sc. 31)	The spec sends “the newest item.” An earlier post in the same week is never emailed and silently ages out.	Track sent posts by GUID and send ANY unsent post from the last 7 days.
8	⚠ No circuit breaker on mass archiving (Sc. 37)	THE MOST SERIOUS GAP FOUND. If engagement tracking silently fails, the monthly job concludes every contact is stale and archives the entire audience. No limit, no sanity check, no undo.	Refuse to archive more than 20% of the audience in one run. Alert instead.

The through-line

Notice that almost none of these are crashes. Every serious scenario in this document fails QUIETLY — a lead vanishes, a digest doesn't send, a newsletter reaches four people instead of three hundred, an entire audience is archived by a job doing exactly what it was told. In every case, the logs say success. That is what “brittle” actually means, and it is what this suite exists to prevent. A system that fails loudly is fine. A system that fails silently costs Rachel a client six weeks from now, and nobody ever knows why.